



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Eduardo Sebastián Moran Bautista **Carné:** 1119425

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. **¿Qué es una API REST?** Es una interfaz de programación de aplicaciones que sigue los principios de arquitectura REST (Represent transferencia de estado representacional), permitiendo la comunicación entre sistemas a través del protocolo HTTP.
2. **¿Qué es un endpoint?** Es la ruta específica o URL final (por ejemplo, /pet/{petId}) donde una API puede acceder a los recursos que un cliente solicita.
3. **¿Qué diferencia existe entre una URL y un endpoint?** La URL es la dirección completa de un servidor (ej. <https://petstore3.swagger.io>), mientras que el endpoint es la ruta específica dentro de esa URL que apunta a un recurso determinado.
4. **¿Qué función cumplen los métodos HTTP?**
 - **GET:** Obtiene o consulta recursos.
 - **POST:** Crea un nuevo recurso.
 - **PUT / PATCH:** Actualizan recursos (PUT reemplaza el recurso completo; PATCH lo actualiza parcialmente).
 - **DELETE:** Elimina un recurso.
5. **¿Qué es JSON y por qué es común en las APIs?** *JavaScript Object Notation* es un formato ligero de intercambio de datos basado en texto, fácil de leer para los humanos y de parsear para las máquinas.
6. **¿Qué es un código de estado HTTP?** Códigos numéricos de tres dígitos que indican el resultado de una solicitud. Ejemplos: 200 (OK / Éxito), 404 (Not Found / No encontrado), 500 (Internal Server Error / Error del servidor).
7. **¿Qué información se encuentra en la respuesta de una API?** El cuerpo (body) con los datos solicitados (frecuentemente en JSON), los encabezados (headers) y el código de estado HTTP.
8. **¿Qué ventajas tiene utilizar Postman?** Permite probar, automatizar, depurar y organizar solicitudes de API antes de escribirlas en el código de producción.
9. **¿Qué es Swagger UI?** Es una herramienta visual de código abierto que genera documentación interactiva para APIs REST, permitiendo probar los endpoints directamente desde el navegador.
10. **¿Qué función cumple Postman?** Actúa como un cliente HTTP avanzado para diseñar, construir, probar y documentar APIs de forma estructurada mediante colecciones.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet



- Identifique un endpoint que permita consultar una mascota por ID.

GET /pet/{petId} Find pet by ID.

- Indique el método HTTP utilizado.
R// GET
- Indique qué parámetro necesita la solicitud.
R// (petID) identificador de la mascota ya sea entero o string
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.

https://petstore3.swagger.io/api/v3/pet/1119425

GET https://petstore3.swagger.io/api/v3/pet/1119425

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 9 Test Results

500 Internal Server Error • 17 ms • 501 B

Investigate server Diagnose Decline

{ } JSON Preview Debug with AI

code 500

message There was an error processing your request. It has been logged (ID: 0471d490cd2f6277)

- Explique con sus palabras qué información contiene la respuesta.

Error del servidor

Evidencia: captura de Swagger mostrando la operación y su respuesta.

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1119425' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1119425

Server response

Code Details

500 Error: response status is 500

Response body

```
{
  "code": 500,
  "message": "There was an error processing your request. It has been logged (ID: 5992b4869593577b)"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 110
content-type: application/json
date: Wed, 30 Sep 2026 13:42:38 GMT
server: Jetty(9.4.53.v20231005)
```



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Parameters

No parameters

Request body required

application/json

Create a new pet in the store

Edit Value | Schema

```
{  "id": 11194,  "name": "nala",  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

Execute

Clear



Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 11194,
    "name": "nala",
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
500	Error: response status is 500

Response body

```
{
  "code": 500,
  "message": "There was an error processing your request. It has been logged (ID: 9c378718d809e6e1)"
}
```

https://petstore3.swagger.io/api/v3/pet

GET https://petstore3.swagger.io/api/v3/pet

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
9      "string"
10    ],
11    "tags": [
12      {
13        "id": 0,
14        "name": "string"
15      }
16    ],
17    "status": "available"
18  }
```

Body Cookies Headers 9 Test Results 405 Method Not Allowed • 43 ms • 434 B •

★ Change method to POST Fix Decline

{ } JSON Preview Debug with AI

code	405
message	HTTP 405 Method Not Allowed

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.



- ¿Qué método HTTP utilizó y por qué?
R// PUT
- ¿Qué datos fueron enviados en el body? ID 11194 , “nala”
- ¿Qué código HTTP devolvió la API?
500
- ¿Cómo comprobó que el cambio realmente ocurrió?

PUT /pet Update an existing pet.

Update an existing pet by id.

Parameters

No parameters

Request body required

application/json

Update an existent pet in the store

Edit Value | Schema

```
{
  "id": 11194,
  "name": "nala",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

```
-d {
  "id": 11194,
  "name": "nala",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code	Details
500	Error: response status is 500

Response body

```
{
  "code": 500,
  "message": "There was an error processing your request. It has been logged (ID: e211357a904f98a7)"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 110
content-type: application/json
date: Wed,30 Sep 2026 14:03:16 GMT
server: Jetty(9.4.53.v20231009)
```

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.



- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.